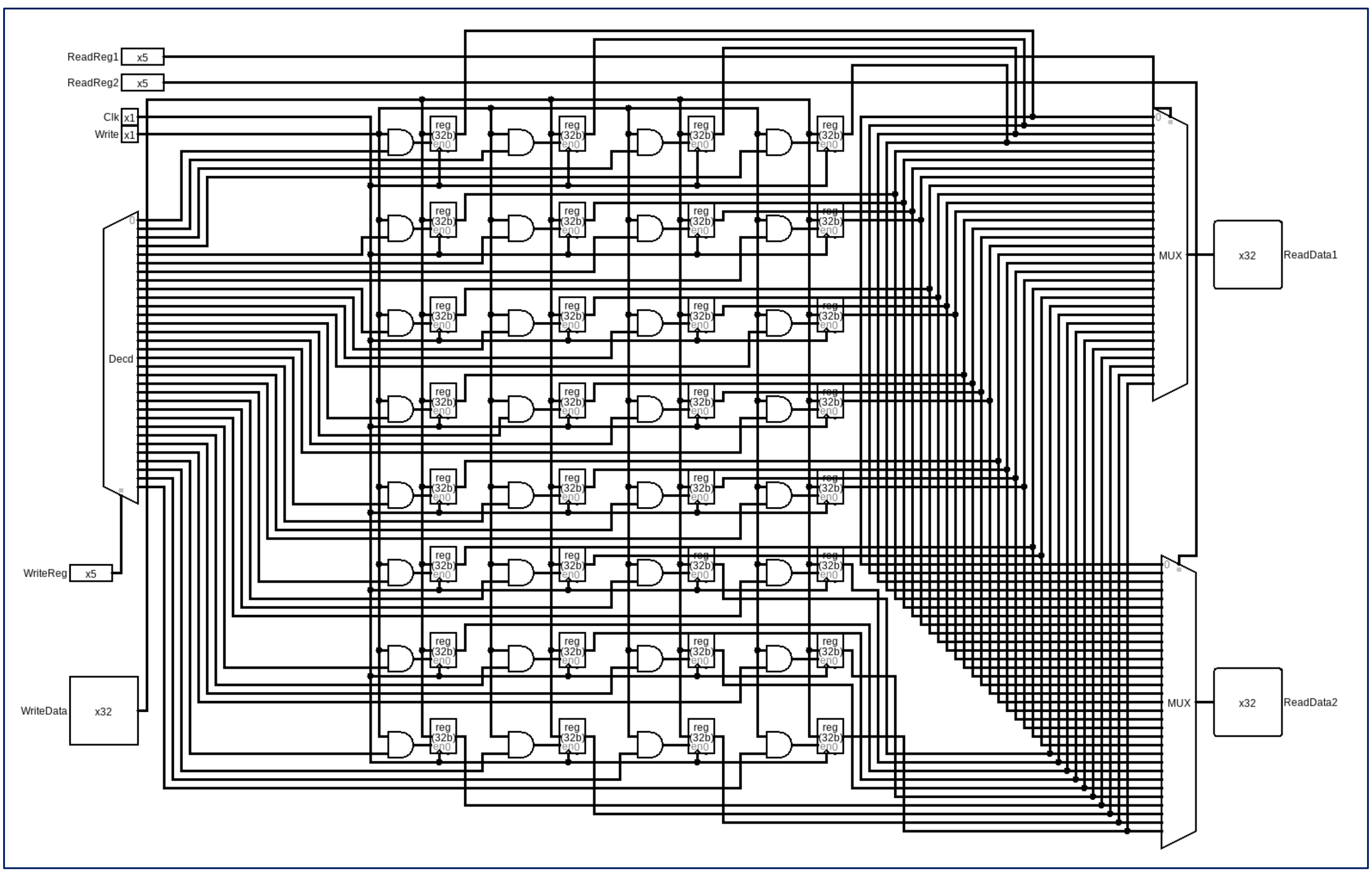
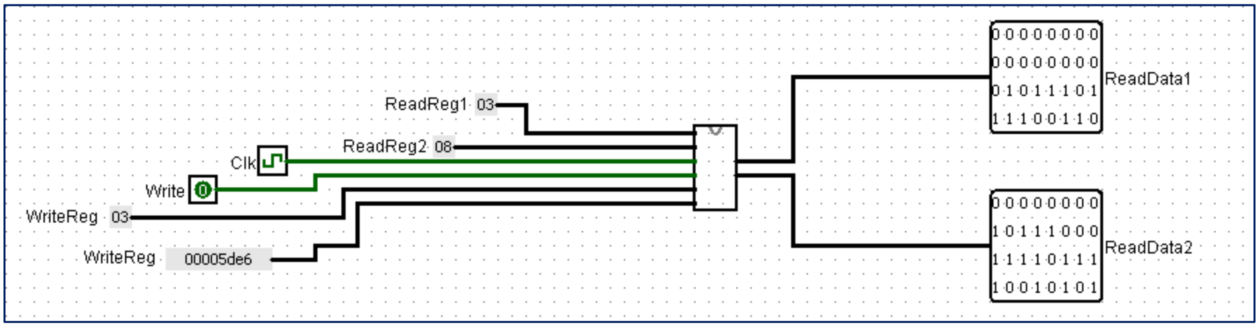
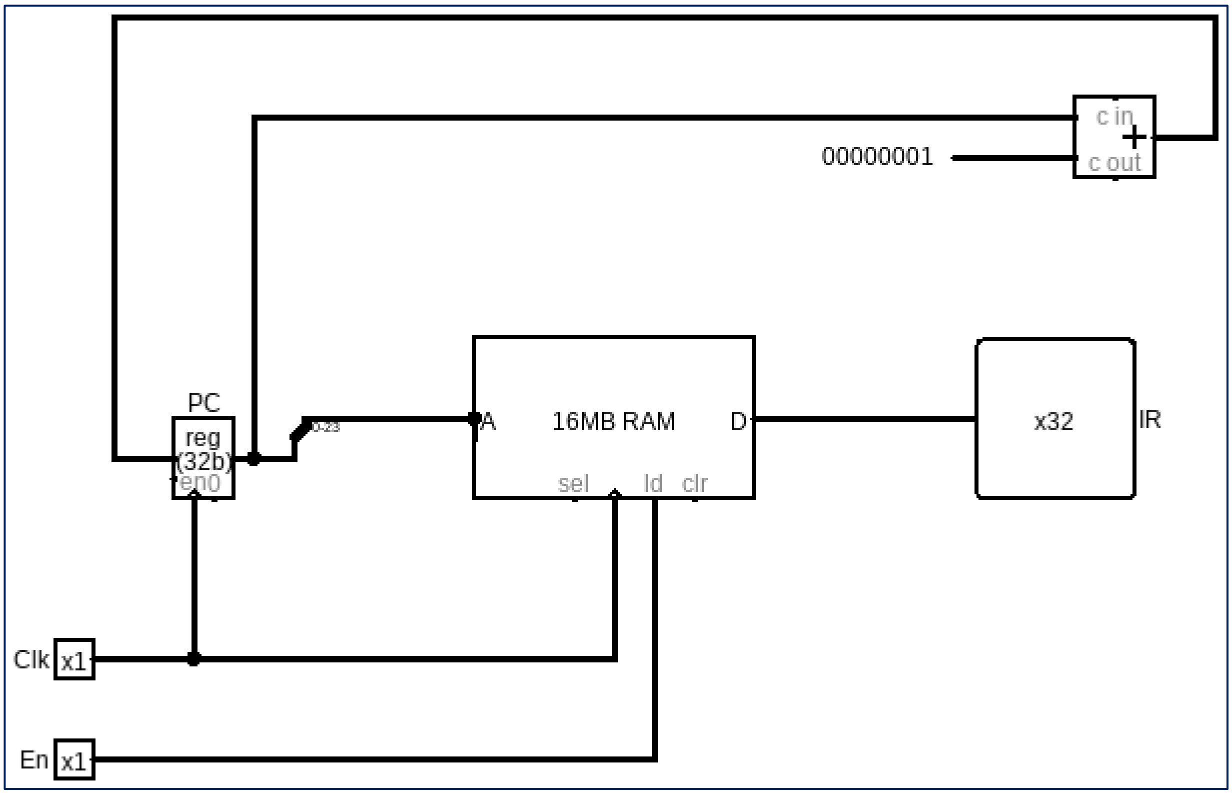
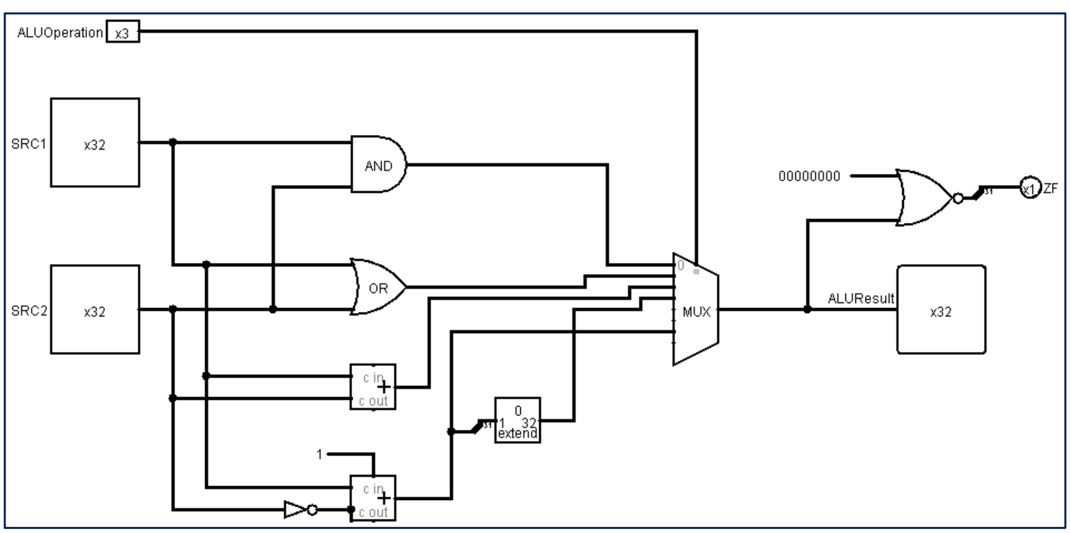
```
.data
        msg1: .asciz "Enter value of n: "
        msg2: .asciz "Factorial: "

.text
.globl main

main:
        # n value input msg
        li a7, 4
        la a0, msg1
        ecall

        # n value
        li a7, 5
        ecall
        mv t0, a0

        add a0, zero, t0        # moving the number(arg) to a0
        jal FACT
        add t1, zero, a0        # moving factorial to t1

        # print FACT msg
        li a7, 4
        la a0, msg2
        ecall

        # print factorial
        li a7, 1
        mv a0, t1
        ecall

        # exit normal termination
        li a7, 10
        ecall

.end main

.globl FACT

.ent FACT

FACT:
        # Stack pointer
        addi sp, sp, -8 # adjust stack pointer
        sw ra, 4(sp)    # save Return Address
        sw a0, 0(sp)    # sace argument n

        # Test for n < 1
        slti t0, a0, 1
        beq t0, zero, L1        # if n >= 1, go to L1
        addi a0, zero, 1        # return 1
        addi sp, sp, 8          # pop item of the stack
        jr ra

L1:
        # Main recursion part
        addi a0, a0, -1         # if n >= 1, arg gets (n - 1)
        jal FACT                # call fact with (n - 1)
        lw t1, 0(sp)            # return from jal, restore arg of n
        lw ra, 4(sp)            # restore the return address
        addi sp, sp, 8          # adjust stack pointer to pop items
        mul a0, t1, a0          # calculate n * fact(n - 1)
        jr ra                   # return to caller

.end FACT
```

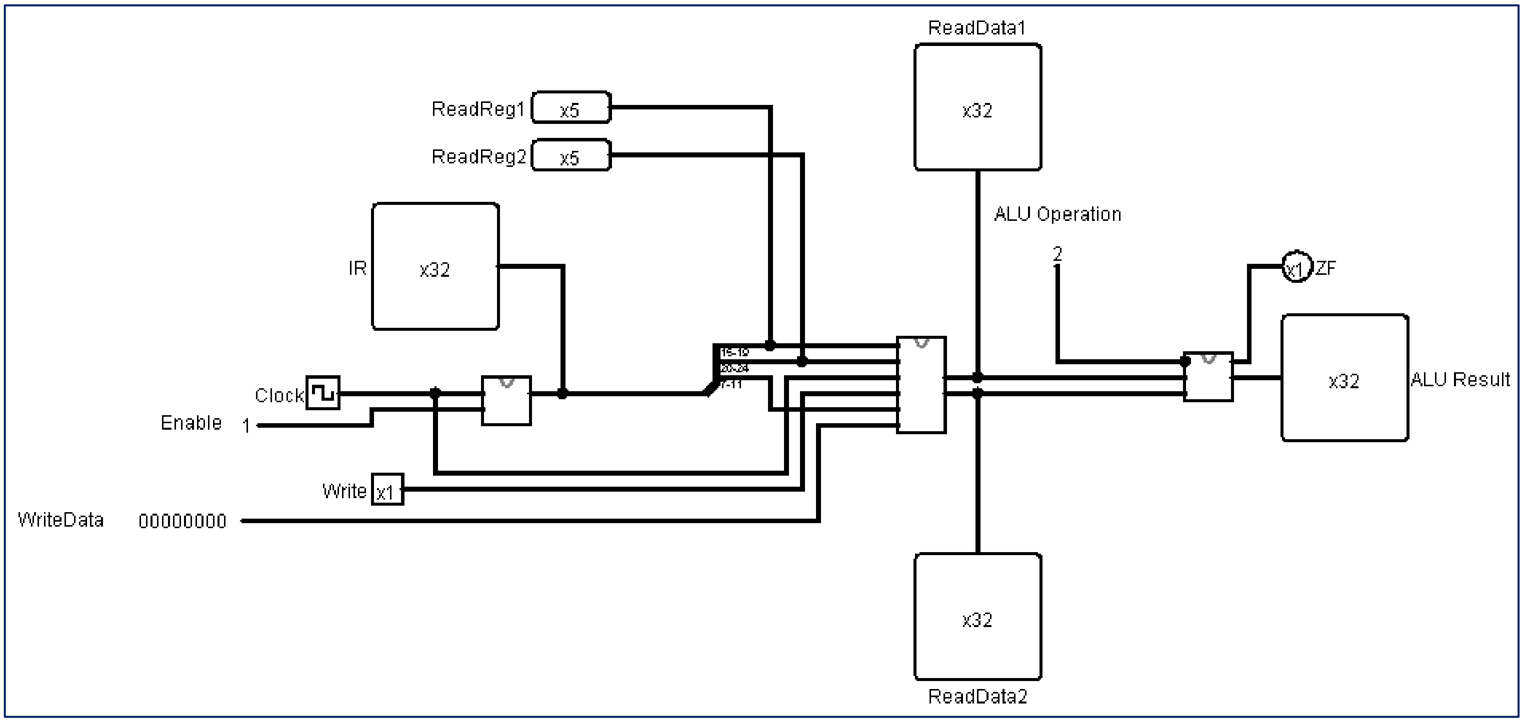
```
Enter value of n: 11
Factorial: 39916800
-- program is finished running (0) --
```

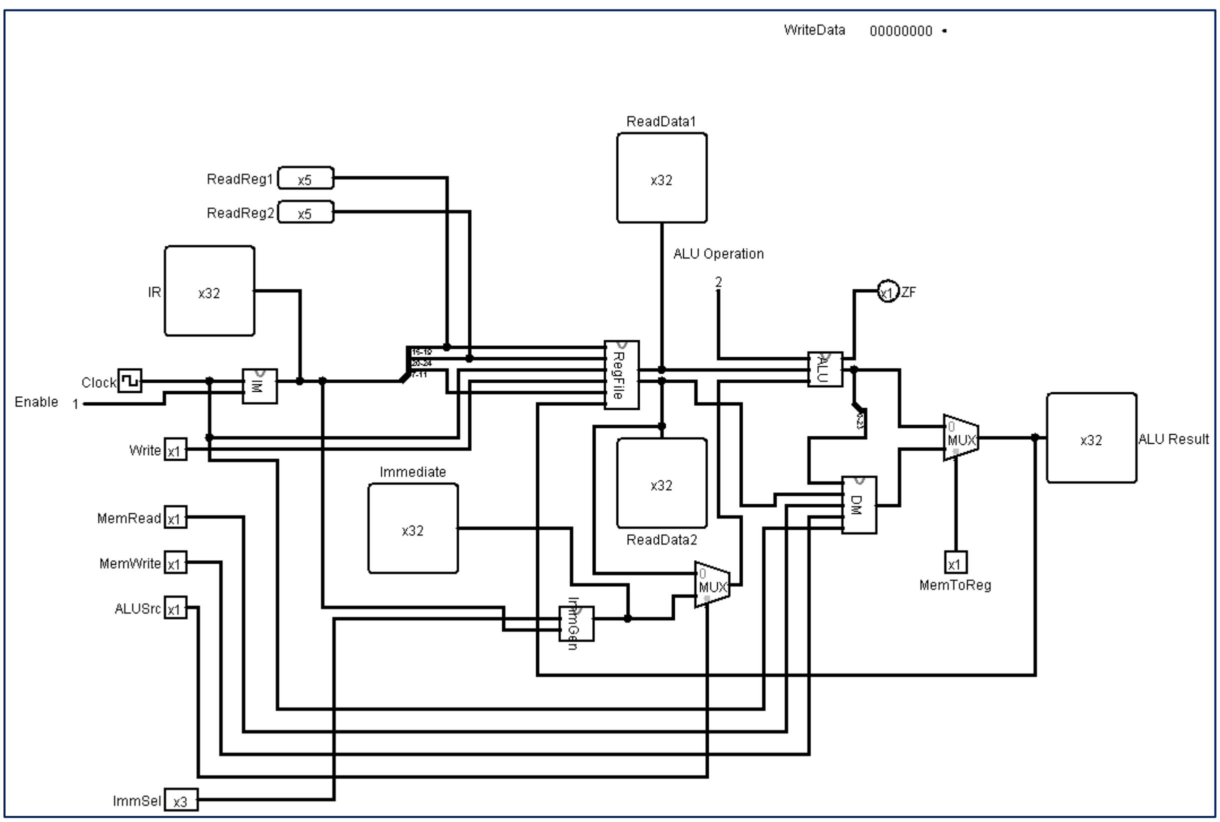



ReadReg1 03
ReadReg2 08
Clk
Write
WriteReg 03
WriteReg 00005de6
ReadData1
ReadData2
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 1 0 1 1 1 0 1
1 1 1 0 0 1 1 0
0 0 0 0 0 0 0 0
1 0 1 1 1 0 0 0
1 1 1 1 0 1 1 1
1 0 0 1 0 1 0 1


c in
c out
00000001
PC
reg
(32b)
en0
A
16MB RAM
D
sel
ld
clr
x32
IR
Clk
x1
En
x1


ALUOperation
x3
SRC1
x32
SRC2
x32
AND
OR
c in
c out
1
extend
MUX
00000000
ZF
ALUResult
x32


ReadData1
x32
ReadReg1
x5
ReadReg2
x5
ALU Operation
IR
x32
2
x1
ZF
Clock
Enable
1
x32
ALU Result
Write
x1
WriteData
00000000
x32
ReadData2


WriteData
00000000
ReadData1
x32
ReadReg1
x5
ReadReg2
x5
ALU Operation
IR
x32
2
x1
ZF
Clock
Enable
1
IM
RegFile
ALU
Write
x1
MUX
x32
ALU Result
Immediate
x32
ReadData2
DM
MemRead
x1
MemWrite
x1
MUX
x1
MemToReg
ALUSrc
x1
ImmGen
ImmSel
x3